

Middle East Technical University
Department of Computer Engineering
Wireless Systems, Networks and Cybersecurity (WINS) Laboratory



Phase 2: COVERT CHANNEL IP Packet Order

CENG519 Network Security
2024-2025 Spring
Term Project Report

Prepared by
Mohamed Eldeeb
Student ID: 2643120
mohamed.eldeeb@metu.edu.tr
Computer Engineering
15 April 2025

1 Introduction

This is the report for the second phase of the term project for Network Security. A covert channel based on IP packet order was implemented. The implementation is based on the method introduced in [1] with simplified behavior.

2 Background and Related Work

2.1 Background

2.1.1 Covert Channel

A covert channel is a form of steganography that relies on an existing communication channel called the overt channel. The purpose of a covert channel is to establish communication in a stealthy manner without disrupting the overt channel's operation. This can be achieved by modulating changes in the properties of the overt communication as data, or by otherwise storing covert information in unused fields in the packets. Examples of covert channels include storage-based channels, which may modify the fields of a packet's header, and time-based channels, which rely on the transmission properties (time between packets, inter-packet jitter, etc.) rather than the data being transmitted.

2.1.2 Packet Order

In the OSI network model, packet arrival order is maintained in connection-oriented networks by the transport layer. TCP, for example, maintains a window for received packets for various reasons, one of which to re-order the packets if they arrive out of order. This is done by checking the sequence number (SEQ) of packets. Thus, if packet reordering occurs in the network before arrival, the application layer of the receiver will still receive the packets in their desired order.

2.2 Related Work

A covert channel design based on packet order is introduced in [1]. The channel exploits the packet reordering phenomenon as a natural occurrence in the Internet. Some of the reasons for packet reordering reviewed in the literature are local parallelism, multi-path forwarding routes, load balancers, and TCP retransmissions. Since the transport layer ensures reception in the order sent, this channel assumes the covert channel receiver will be at the network layer in order to interpret the order of the packets as they arrive.

The channel designed by El-Atawy et al in [1] is based on assigning symbols to the different permutations of a set of reordered packets. Some key parameters are defined as follows:

- k , codeword size, or the number of packets in a permutation,
- l , the number of permutations used to represent the symbols, and
- bpS , the number of bits per symbol.

It should be noted that the inequality $l \leq k!$ has to be satisfied, as $k!$ is the total number of possible permutations of a set of k items.

Symbols represent a binary string of fixed size. The number of bits per symbol (bpS) is equal to $\log_2(l)$. Since it is easier to represent binary data in discrete units, selecting bpS as an integer first would be preferred, and would result in choosing l as 2^{bpS} .

Based on the analysis in [1], the covert channel’s capacity can be calculated using the number of packets transmitted in the overt channel (N) as

$$C = bpS \cdot N/k \quad (1)$$

For example, for the set of parameters $k = 4$ and $bpS = 2$, this results in a capacity of 0.5 bits per overt packet.

Another important aspect of the design is the permutation set used for representing the symbols. Some of the suggestions in [1] are choosing the permutations to be uniformly distributed, and to only select even permutations (permutations that need an even number of element swaps starting from the first order). These suggestions allow for advanced capabilities such as error detection and correction.

3 Implementation

The covert sender and receiver are implemented in the middle box environment designed for the Network Security course ([2]). A simplified version of the design made by El-Atawy et al that does not feature error correction was implemented.

The implementation requires the sender and receiver to agree on the parameters (k and bpS) beforehand. The permutations are selected to be evenly spaced to achieve maximum inter-symbol distance. This assists in error detection, and can be leveraged for error correction as in [1], although it is not implemented. One disadvantage of the permutations being uniformly distributed is that it might increase the probability of detection, as orders that are not naturally occurring will appear a lot (e.g., a perfect reverse order: 4, 3, 2, 1). However, for smaller codeword sizes, this effect is not that significant.

4 Results and Discussion

4.1 Methodology

The go-processor implemented in phase 1 of the project was run with a delay mean of 1ms. The overt data was a string of capital English letters (A to Z), and was sent letter by letter, such that each overt packet carries a payload of 1 byte. The covert data was the string "The quick brown fox jumps over the lazy dog." The sender repeated each of the data in a cyclic manner, and the receiver logged the decoded covert data. For each set of parameters, a total of 3000 packets were sent at intervals of 5ms.

4.2 Results

In the test environment, no bit errors (i.e., incorrect permutations) occurred. This does not model real network behavior on the Internet, but will suffice for the purposes of this report.

The results in Figure 1 show the channel capacity (in covert bits transmitted) for the performed test over different k and bpS values. This matches the theoretical rates as shown in 2, as covert bits per overt packet transmission.

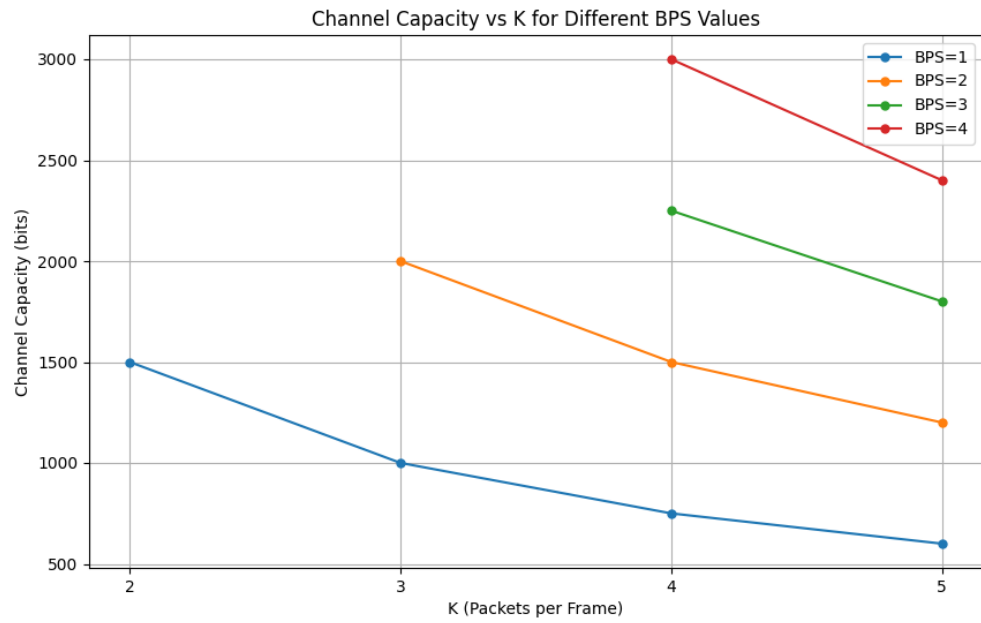


Figure 1 . Channel capacity results

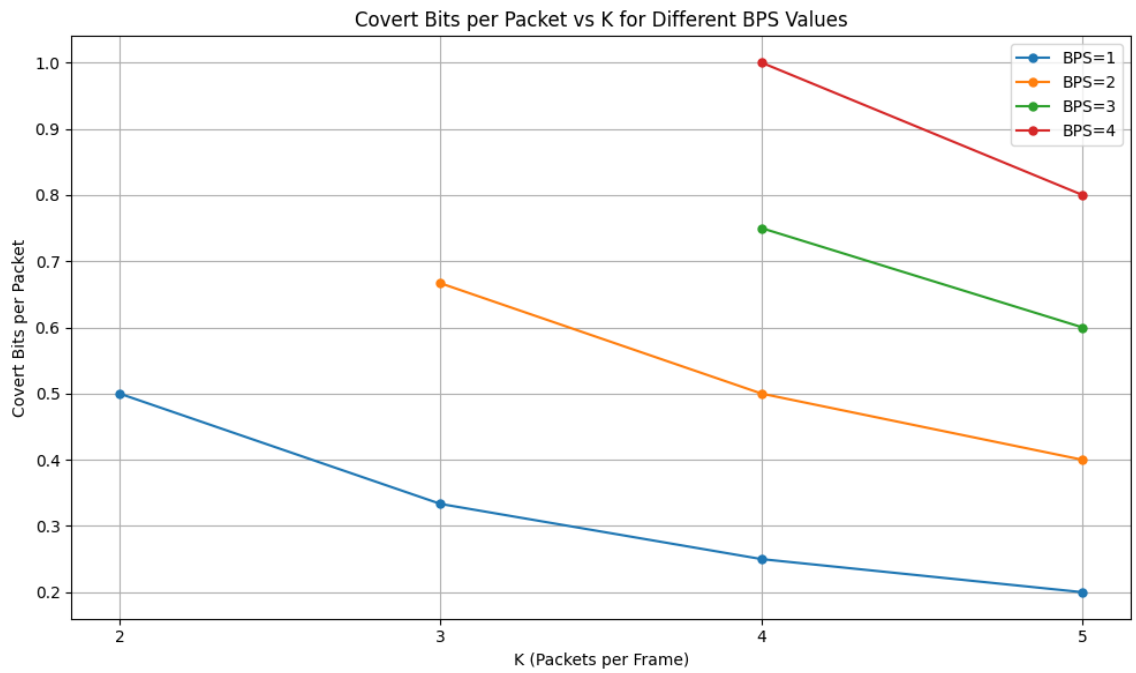


Figure 2 . Channel capacity results (bit rate)

References

- [1] A. El-Atawy and E. Al-Shaer, “Building Covert Channels over the Packet Reordering Phenomenon,” in *IEEE INFOCOM 2009*. IEEE, pp. 2186--2194. [Online]. Available: <https://ieeexplore.ieee.org/document/5062143/>
- [2] E. Onur, “Middlebox.” [Online]. Available: <https://github.com/cengwins/middlebox>